B. Marshalling → envoi → unmarshalling
C. Cryptage obligatoire des données
D. Mise en cache automatique des appels

17. Dans un système réparti, comment l'IDL, le marshalling et la représentation intermédiaire des données interagissent-ils ?
A. L'IDL remplace le marshalling, qui n'est plus nécessaire lorsqu'une représentation intermédiaire est utilisée.
B. Le marshalling s'appuie sur la représentation intermédiaire définie grâce à l'IDL pour convertir les données dans un format commun transmissible entre machines hétérogènes.
C. L'IDL effectue directement le marshalling tandis que la représentation intermédiaire convertit automatiquement les messages réseau.
D. La représentation intermédiaire est utilisée uniquement dans le registre RMI, indépendamment de l'IDL et du marshalling.

18. Dans RMI, que réalise exportObject() ?
A. Supprime l'objet local
B. Sérialise l'objet dans un fichier
C. Rend l'objet accessible à distance en créant sa souche serveur
D. Redémarre le registre

19. Que retourne Naming.lookup() ?
A. Une connexion socket
B. L'adresse du registre
C. La souche (stub) d'un objet distant enregistré
D. Un fichier de configuration

20. En RMI, que fait l'opération de binding ?
A. Elle associe un objet distant à un nom dans le registre RMI
B. Elle envoie les paramètres d'une méthode au serveur
C. Elle supprime un objet du serveur
D. Elle convertit un objet en flux de données

Exercice 2:

À l'occasion du White Friday, un restaurant souhaite développer une application client–serveur en Java utilisant des sockets TCP afin d'automatiser l'attribution d'une commande secrète offerte au client. Au démarrage, le serveur initialise un menu composé d'une liste de plats. Chaque plat est défini par un identifiant entier (ID), un nom (par exemple « Pâtes Bolognaises ») et un prix. Une fois cette initialisation effectuée, le serveur se met en attente de la connexion d'un client. Lorsqu'un client se connecte, le serveur lui envoie le message suivant : « Choisissez votre plat secret dont le code est compris entre 0 et 5 ». Le client sélectionne alors aléatoirement un identifiant parmi la liste reçue et l'envoie au serveur. À la réception du numéro choisi, le serveur recherche le plat correspondant et renvoie au client l'objet représentant ce plat. Enfin, après réception du plat, le client affiche avec (*System. out.println*) un

555F

AVIS IMPORTANT AUX ETUDIANTS

1. Chacune des feuilles de votre copie doit comporter une étiquette code à barres placée à l'endroit indiqué «coller ici votre code à barres».
2. Une copie d'examen comporte une seule «feuille principale» et des «feuilles suites ». sur chacune de vos feuilles , le code à barres est obligatoire.
3. Cette feuille d'examen est strictement personnelle . Elle ne doit comporter aucun signe distinctif.
Elle doit etre ecrite en noir et / ou bleu.
4. Le non respect de l'une de ces recommandations peut faire attribuer la note ZERO à l'épreuve.

Institut Supérieur d'Informatique & de Mathématique de Monastir

NOTE

Coller ici votre code à barre

00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20

00	25	50	75

Institut Supérieur d'Informatique et de Mathématiques de Monastir

ISIMM

Devoir Surveillé– S1 – 2025/2026

Filière : *3ème LI*	**Matière : Développement d'applications réparties**	**Enseignant : Marwa Thabet**
Date : 22/ 11 / 2025		**Documents autorisés : Non**
Durée de l'examen : 1h00		**Nombre de pages : 08**

Nom , prénom et signature de l'enseignant correcteur	

Exercice 1: Encercler la ou les lettres correspondantes à la ou les bonnes réponses.

1. Qu'est-ce qu'une application répartie ?
   A. Une application qui s'exécute uniquement sur un seul ordinateur mais utilise plusieurs threads.
   B. Une application qui répartit son interface graphique entre plusieurs écrans.
   C. Une application dont les composants logiciels sont distribués sur plusieurs machines et coopèrent via un réseau.
   D. Une application destinée uniquement aux serveurs cloud.
2. La coopération dans un système réparti :
   A. Implique la gestion de la distribution des communications
   B. se limite à la synchronisation des processus
   C. Désigne la communication et l'interaction entre processus
   D. Peut porter seulement sur les tâches
3. Les sockets sont :
   A. Un mécanisme bidirectionnel de communication interprocessus
   B. Une interface liée à un protocole unique
   C. Une API spécifique à la couche IP uniquement
   D. Un système de fichiers distant
4. Lesquels des énoncés suivants sont corrects ?
   A. IP est largement utilisé pour la communication entre machines
   B. IP garantit la fiabilité
   C. IP fonctionne au-dessus d'autres couches
   D. TCP/IP et UDP/IP sont accessibles via les sockets
5. Quelle est la différence principale entre TCP et UDP ?
   A. TCP est orienté connexion ; UDP est sans connexion
   B. UDP est plus fiable
   C. TCP est toujours plus rapide
   D. UDP utilise le marshalling
6. En UDP, un DatagramPacket représente :
   A. Une connexion persistante
   B. Un paquet autonome contenant données + adresse
   C. Un flux continu
   D. Un tunnel sécurisé
7. Quel objet un serveur TCP Java utilise-t-il pour accepter les connexions ?
   A. DatagramSocket
   B. Socket
   C. ServerSocket
   D. DataInputStream
8. En UDP, quel objet envoie/reçoit les données ?

   A. Socket
   B. DatagramPacket
   C. PrintStream
   D. ServerSocket
9. À quoi sert la classe MulticastSocket ?
   A. À envoyer des messages unicast
   B. À recevoir des messages TCP
   C. À envoyer/recevoir des messages dans un groupe multicast
   D. À créer des connexions persistantes
10. Dans MulticastSocket, joinGroup() permet :
   A. D'envoyer un message au groupe
   B. De rejoindre un groupe multicast pour recevoir ses messages
   C. D'ouvrir une connexion TCP
   D. De sérialiser des objets
11. Quelle différence distingue RPC et RMI ?
   A. RPC invoque des procédures ; RMI invoque des objets distants
   B. RMI fonctionne uniquement en C
   C. RPC supporte l'héritage, RMI non
   D. RMI n'utilise pas de marshalling
12. Comment les souches client et serveur sont-elles générées en RMI ?
   A. Écrites manuellement
   B. Générées automatiquement à partir de l'interface distante
   C. Générées via un serveur HTTP
   D. Fournies par le registre RMI
13. Quel est le rôle de la sérialisation en Java ?
   A. Convertir un objet en flux d'octets pour le stocker ou le transmettre
   B. Copier un objet d'une classe à une autre
   C. Exécuter les méthodes d'un objet à distance
   D. Définir le type d'un objet à l'exécution
14. Le marshalling est :
   A. L'exécution d'une méthode distante
   B. La création des souches
   C. L'emballage/sérialisation des paramètres avant envoi
   D. La compression réseau
15. Dans un environnement RMI, quel est le rôle de la souche (stub) côté client ?
   A. Gérer le stockage local des objets distants
   B. Masquer la communication réseau et représenter l'objet distant localement
   C. Exécuter les méthodes à la place du serveur
   D. Convertir les classes Java en bytecode RMI
16. Que réalise la souche client lorsqu'elle invoque une méthode distante ?
   A. Compilation du code

SUITE

Coller ici votre
code à barre

message de remerciement selon le prix du plat :
– « Merci pour le X qui coûte cher » si le prix du plat dépasse 50 dt ;
– « Le X ne vaut pas le coût » si le prix du plat est inférieur ou égal à 50 dt.
Ici, X correspond au nom du plat.
Note: On suppose que la classe *Plat* est déjà fournie et qu'elle comporte un getter pour chacun de ses attributs.

1) Implémenter la classe serveur tel qu'il accepte plusieurs clients simultanés (threads).
Le serveur doit être installé sur la machine de l'adresse Resto.WhiteFriday.tn sachant que le service de noms doit être activé sur le port numéro 5000.

```
import java.net.*;  import java.io.*;
public class Server {
   public static List<Plat> menu = new ArrayList<>();
   public static void main(String[] args) {
      // Initialisation du menu
      menu.add(new Plat(0, "Pâtes Bolognaises", 22.5));
      menu.add(new Plat(1, "Couscous Royal", 60));
      menu.add(new Plat(2, "Escalope Crème", 48));
      menu.add(new Plat(3, "Pizza 4 Fromages", 32));
      menu.add(new Plat(4, "Poulet Grillé", 55));
      menu.add(new Plat(5, "Lazagne Maison", 28));

      try {
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
      } catch (Exception e){ e.printStackTrace(); }}}
```



```
.....
      // 2. Le client choisit aléatoirement un numéro entre 0 et 5
      Random r = new Random();
      int choix = r.nextInt(6);
      // 3. Envoi du choix
.....
.....
.....
.....
.....
.....

      // 4. Réception de la réponse
.....
.....
.....
.....
.....
.....

      //5.  Affichage du message final selon le prix
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
      } catch (Exception e) {e.printStackTrace(); }
}}
```

```
import java.net.*; import java.io.*;

public class Service extends Thread {
.....
.....
.....
.....
.....
.....
.....
    public void ..... {
      try {
.....
.....
.....
.....
.....
.....
.....
    // 1. Envoi du message initial
.....
.....
.....
.....
.....
.....
.....
.....
    // 2. Réception du choix du client

.....
.....
.....
```



```
.....
.....
Plat selection = menu.get((int) choix);  // Récupération du plat sélectionné de la liste
        // 3.  Envoi du plat
.....
.....
.....
.....
.....
.....
.....

    } catch (Exception e){ e.printStackTrace(); }}

}
```

2) Implémenter la classe client.

```
import java.io.*;
import java.net.*;
public class Client {
  public static void main(String[] args) {
      try {
.....
.....
.....
.....
.....
.....
.....
.....
      // 1. Réception du message initial
.....
.....
.....
.....
```